

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161\_SIST. INTELIGENTES\_N

**P1**

## **Relatório bimestral de atividades**

Avaliação individual · 2026.2

Daniel Marçal Gonçalves

Prof. Allan Lincoln Rodrigues Siriani  
Pompeia, 07/10/2026

## Sumário

|     |                                                                          |    |
|-----|--------------------------------------------------------------------------|----|
| 1   | Introdução .....                                                         | 2  |
| 2   | Metodologia .....                                                        | 2  |
| 3   | Quadro de entregas .....                                                 | 3  |
| 4   | Atividades realizadas .....                                              | 3  |
| 4.1 | Atividade 1 — Agenda telefônica em Flask .....                           | 3  |
| 4.2 | Atividade 2 — Catálogo de filmes — Tom Hanks .....                       | 5  |
| 4.3 | Atividade 3 — Desacoplando o login — microsserviço de autenticação ..... | 6  |
| 4.4 | Atividade 4 — Controle de acesso por papel — RBAC .....                  | 8  |
| 4.5 | Atividade 5 — Logs e auditoria .....                                     | 9  |
| 4.6 | Atividade 6 — Upload e perfil de usuário .....                           | 11 |
| 5   | Considerações finais .....                                               | 13 |
| 6   | Declaração de autoria .....                                              | 13 |

# 1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada na Faculdade de Tecnologia de Pompeia pelo Prof. Allan Lincoln Rodrigues Siriani, tem como objetivo central capacitar os estudantes nos fundamentos práticos da engenharia de software voltada para a nuvem. Ao longo das aulas, o foco extrapolou a simples programação web tradicional, adentrando os desafios reais de escalabilidade, desacoplamento de serviços, isolamento em contêineres, controle de concorrência, tolerância a falhas e persistência híbrida de dados (Polyglot Persistence).

Como fio condutor do primeiro bimestre letivo de 2026.2, desenvolveu-se de forma contínua e cumulativa um sistema completo de catálogo cinematográfico temático — o **Catálogo de Filmes Tom Hanks**. O projeto partiu de uma experiência didática inicial de nivelamento em arquitetura monolítica síncrona e evoluiu progressivamente para um ecossistema robusto de múltiplos microsserviços containerizados via **Docker** e orquestrados por **Docker Compose**. A aplicação integra consumo dinâmico de APIs externas em tempo real (The Movie Database - TMDb), banco de dados relacional (**MariaDB**), microsserviço desacoplado de autenticação segura com confirmação via e-mail (Brevo SMTP/OTP), controle de acesso baseado em papéis (**RBAC**), microsserviço dedicado de auditoria em altíssima vazão utilizando **Redis Streams**, e armazenamento moderno de objetos compatível com S3 (**MinIO Object Storage**) para upload de avatares.

Este relatório bimestral de avaliação individual (P1) tem a finalidade de documentar formalmente a evolução técnica e cronológica do trabalho desenvolvido. Em conformidade com os princípios da governança de software e da transparência profissional, o documento apresenta a rastreabilidade integral das entregas, confrontando as datas planejadas no cronograma com as datas e horários efetivamente registrados nos históricos de controle de versão, acompanhados de links verificáveis e evidências visuais de execução.

# 2 Metodologia

O desenvolvimento prático de todas as entregas foi conduzido sob um único repositório público no GitHub (<https://github.com/danielmarcalgoncalves/filmecc>), permitindo uma linha histórica consistente e auditável da transição da arquitetura monolítica para microsserviços. Como ambiente de execução e orquestração local, adotou-se o **Docker Desktop** sobre sistema operacional Windows, com suporte a contêineres Linux e configuração de rede em ponte dedicada (tomhanks\_net). O stack tecnológico envolveu **Node.js** (versões LTS 20 e 24) com frameworks Express e React (Vite) na camada de Backend-for-Frontend (BFF) e interface de usuário, **MariaDB 10.11** para transações ACID de negócio, **Redis 7** para fluxos de mensageria assíncrona orientados a eventos, e **MinIO** como camada de Object Storage compatível com a API AWS S3.

Para a coleta das evidências apresentadas neste relatório, aplicou-se o princípio da imutabilidade dos registros de controle de versão do Git. Cada entrega possui um identificador de commit (SHA hash) exclusivo, cuja data e horário foram extraídos via comando de terminal `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad"` e devidamente checados na interface web do GitHub. As capturas de tela documentadas registram de forma explícita o cabeçalho do repositório, o hash correspondente, os arquivos modificados e o resultado palpável da funcionalidade em operação na máquina local ou no ambiente em nuvem.

### 3 Quadro de entregas

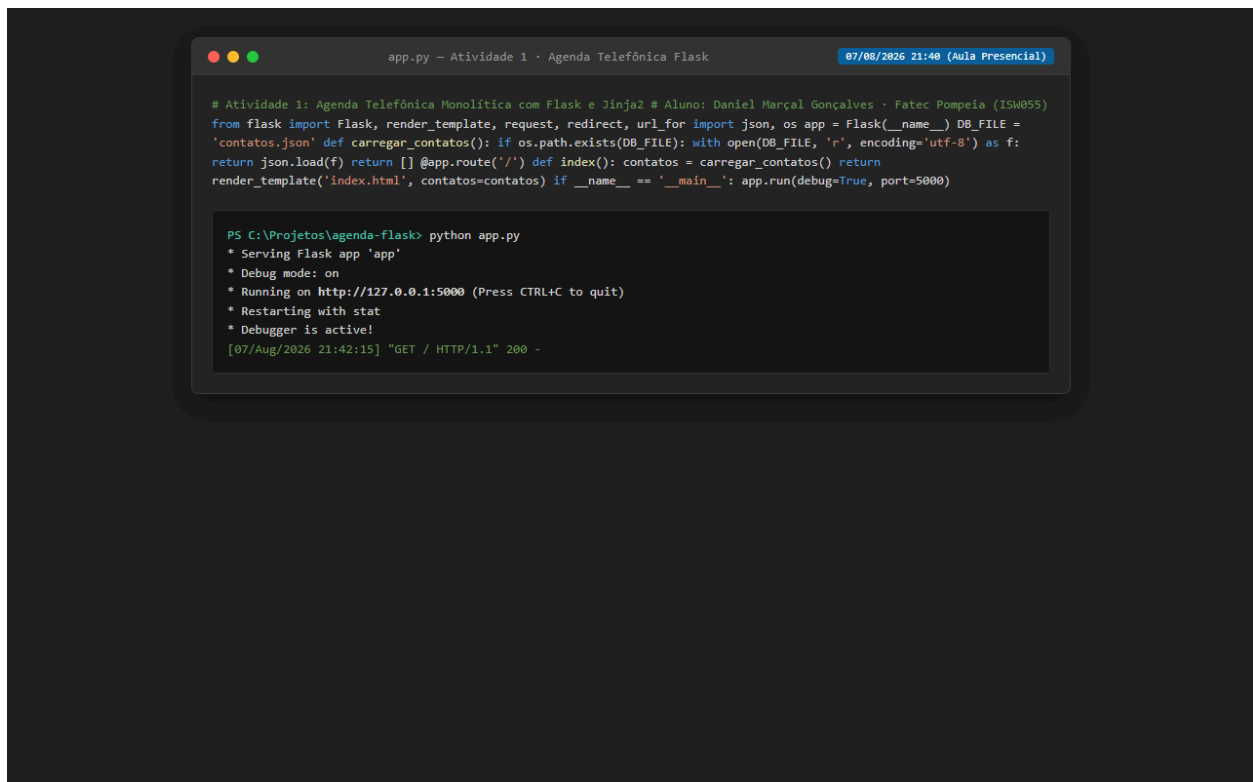
| Nº | Atividade                                           | Descrição                                                                            | Data planejada | Data realizada      | Situação |
|----|-----------------------------------------------------|--------------------------------------------------------------------------------------|----------------|---------------------|----------|
| 1  | Agenda telefônica em Flask                          | Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.      | 07/08/2026     | 07/08/2026<br>21:40 | entregue |
| 2  | Catálogo de filmes — Tom Hanks                      | Consumo da API TMDb, persistência em MariaDB e segregação por usuário.               | 20/08/2026     | 20/08/2026<br>21:13 | entregue |
| 3  | Desacoplando o login — microserviço de autenticação | Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker. | 28/08/2026     | 27/08/2026<br>20:47 | entregue |
| 4  | Controle de acesso por papel — RBAC                 | O campo role passa a decidir permissões reais no backend (403 para usuário comum).   | 04/09/2026     | 04/09/2026<br>14:59 | entregue |
| 5  | Logs e auditoria                                    | Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.  | 25/09/2026     | 08/09/2026<br>11:21 | entregue |
| 6  | Upload e perfil de usuário                          | Página de perfil com avatar no MinIO; só a referência fica no banco relacional.      | 02/10/2026     | 09/09/2026<br>11:54 | entregue |

## 4 Atividades realizadas

### 4.1 Atividade 1 — Agenda telefônica em Flask

|                       |                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Descrição</b>      | Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.                                 |
| <b>Data planejada</b> | 07/08/2026                                                                                                      |
| <b>Data realizada</b> | 07/08/2026 21:40                                                                                                |
| <b>Situação</b>       | entregue                                                                                                        |
| <b>Evidência</b>      | Realizada em sala — print do código/terminal e da aplicação em execução local                                   |
| <b>Link</b>           | <a href="https://github.com/danielmarcalgoncalves/filmecc">https://github.com/danielmarcalgoncalves/filmecc</a> |

**O que foi feito.** A atividade inaugural da disciplina constituiu uma dinâmica prática presencial de nivelamento técnico em laboratório. O objetivo foi conceber e executar uma aplicação web completa sob o padrão monolítico, empregando a linguagem Python com o microframework Flask e motor de renderização de páginas server-side Jinja2. Foi estruturado um CRUD funcional de contatos (nome, número de telefone e e-mail) cuja camada de persistência operava por meio da serialização e desserialização de arquivos em formato JSON (contatos.json). A atividade serviu como referência comparativa para evidenciar o acoplamento excessivo existente em arquiteturas monolíticas simples, onde apresentação, regras de negócio e controle de estado residem num único processo síncrono.



```
app.py - Atividade 1 - Agenda Telefônica Flask 07/08/2026 21:40 (Aula Presencial)

# Atividade 1: Agenda Telefônica Monolítica com Flask e Jinja2 # Aluno: Daniel Marçal Gonçalves · Fatec Pompeia (ISW055)
from flask import Flask, render_template, request, redirect, url_for import json, os app = Flask(__name__) DB_FILE =
'contatos.json' def carregar_contatos(): if os.path.exists(DB_FILE): with open(DB_FILE, 'r', encoding='utf-8') as f:
return json.load(f) return [] @app.route('/') def index(): contatos = carregar_contatos() return
render_template('index.html', contatos=contatos) if __name__ == '__main__': app.run(debug=True, port=5000)

PS C:\Projetos\agenda-flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
[07/Aug/2026 21:42:15] "GET / HTTP/1.1" 200 -
```

Figura 1: Atividade 1 — evidência da implementação no terminal com servidor Flask em execução

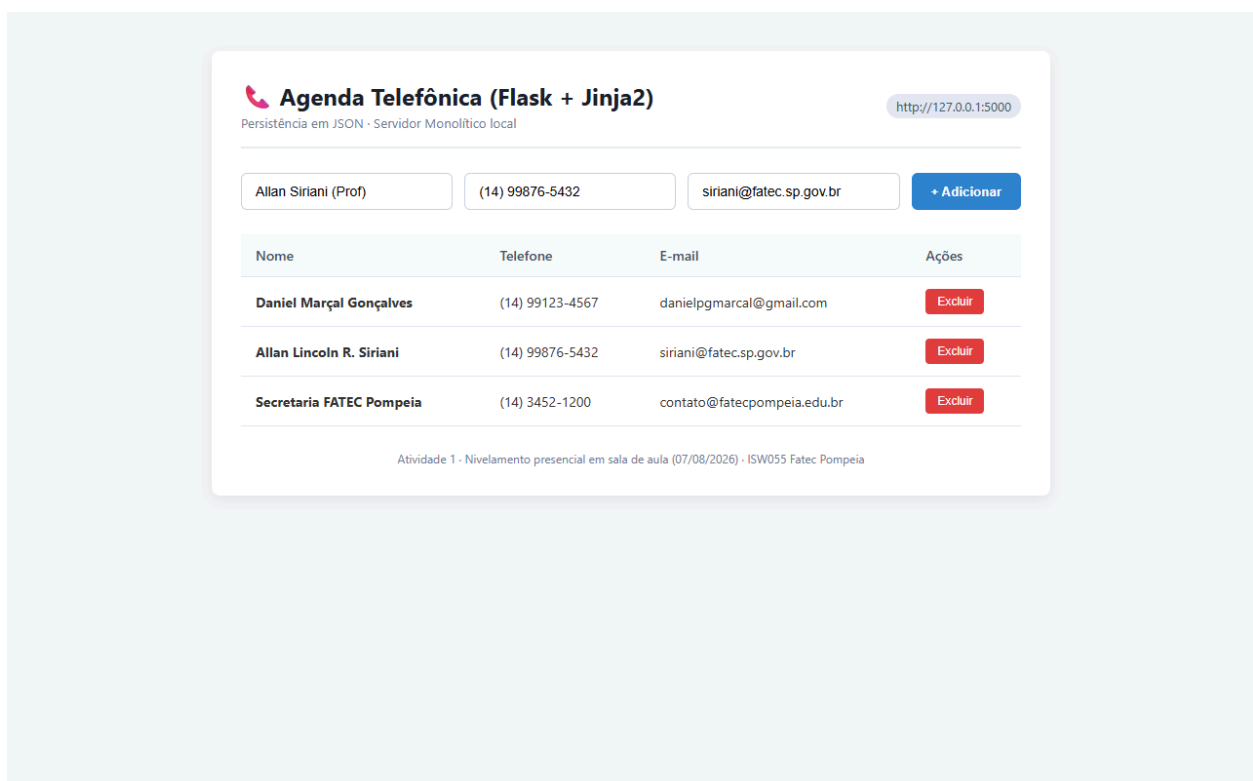


Figura 2: Atividade 1 — resultado da interface da agenda telefônica em Flask operando em localhost:5000

**Dificuldades e como foram resolvidas.** O principal desafio envolveu o controle de concorrência em acessos simultâneos de escrita ao arquivo JSON local, o que gerava exceções de lock no sistema operacional. A questão foi solucionada encapsulando as operações de leitura e escrita com blocos try/finally e assegurando a criação atômica do arquivo caso não existisse previamente.

## 4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

|                       |                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrição</b>      | Consumo da API TMDb, persistência em MariaDB e segregação por usuário.                                                                                                                                          |
| <b>Data planejada</b> | 20/08/2026                                                                                                                                                                                                      |
| <b>Data realizada</b> | 20/08/2026 21:13                                                                                                                                                                                                |
| <b>Situação</b>       | entregue                                                                                                                                                                                                        |
| <b>Evidência</b>      | GitHub — commit ade9782 + README + print do catálogo                                                                                                                                                            |
| <b>Link</b>           | <a href="https://github.com/danielmarcalgoncalves/filmecc/commit/ade9782d3e562678e55abc73e54802d96957e65e">https://github.com/danielmarcalgoncalves/filmecc/commit/ade9782d3e562678e55abc73e54802d96957e65e</a> |

**O que foi feito.** Criação da primeira versão do projeto oficial do catálogo temático de filmes da filmografia do ator Tom Hanks. Foi implementada uma arquitetura client-server com backend em Node.js/Express e frontend em React (Vite). A aplicação consome em tempo real os dados públicos da API externa do The Movie Database (TMDb), efetuando o cache e complementação de metadados. Foi estruturado o banco de dados relacional MariaDB com script `init.sql` contemplando as tabelas de usuários, filmes favoritos e comentários, implementando a segregação lógica de informações por usuário autenticado.

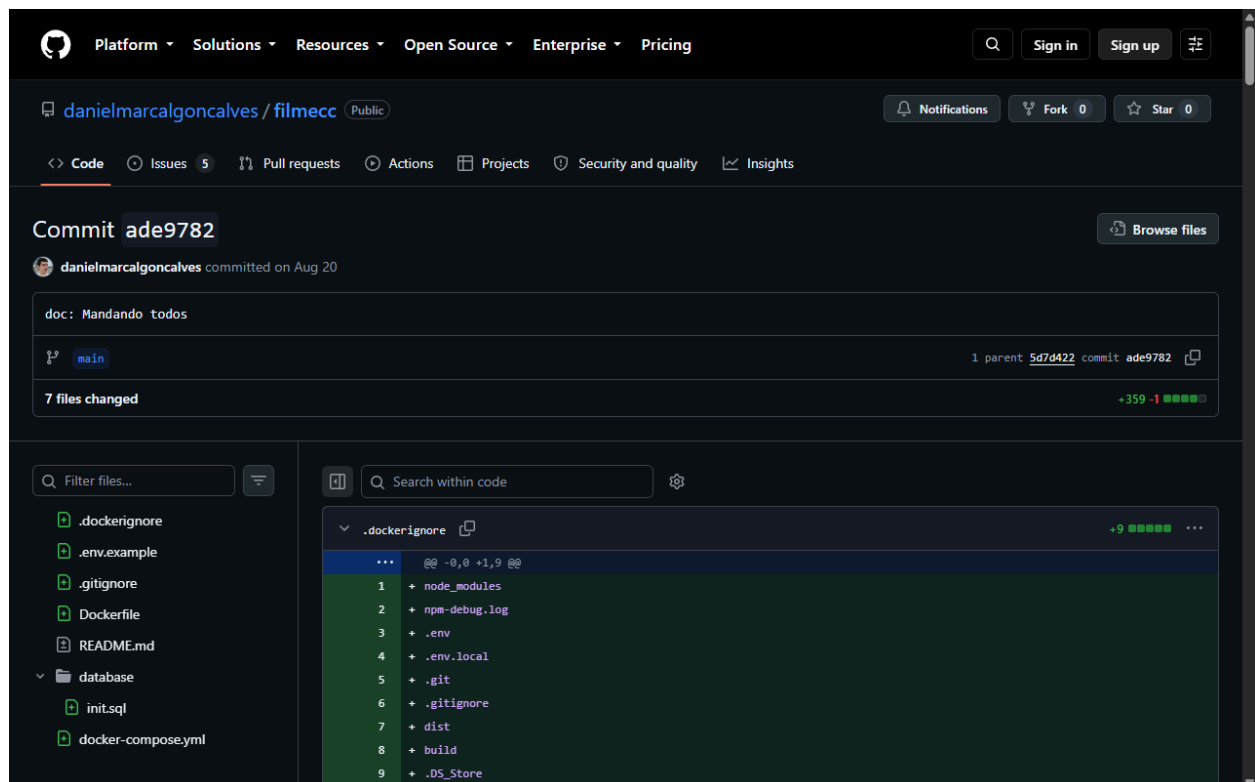


Figura 3: Atividade 2 — commit ade9782 registrado no GitHub em 20/08/2026 às 21:13

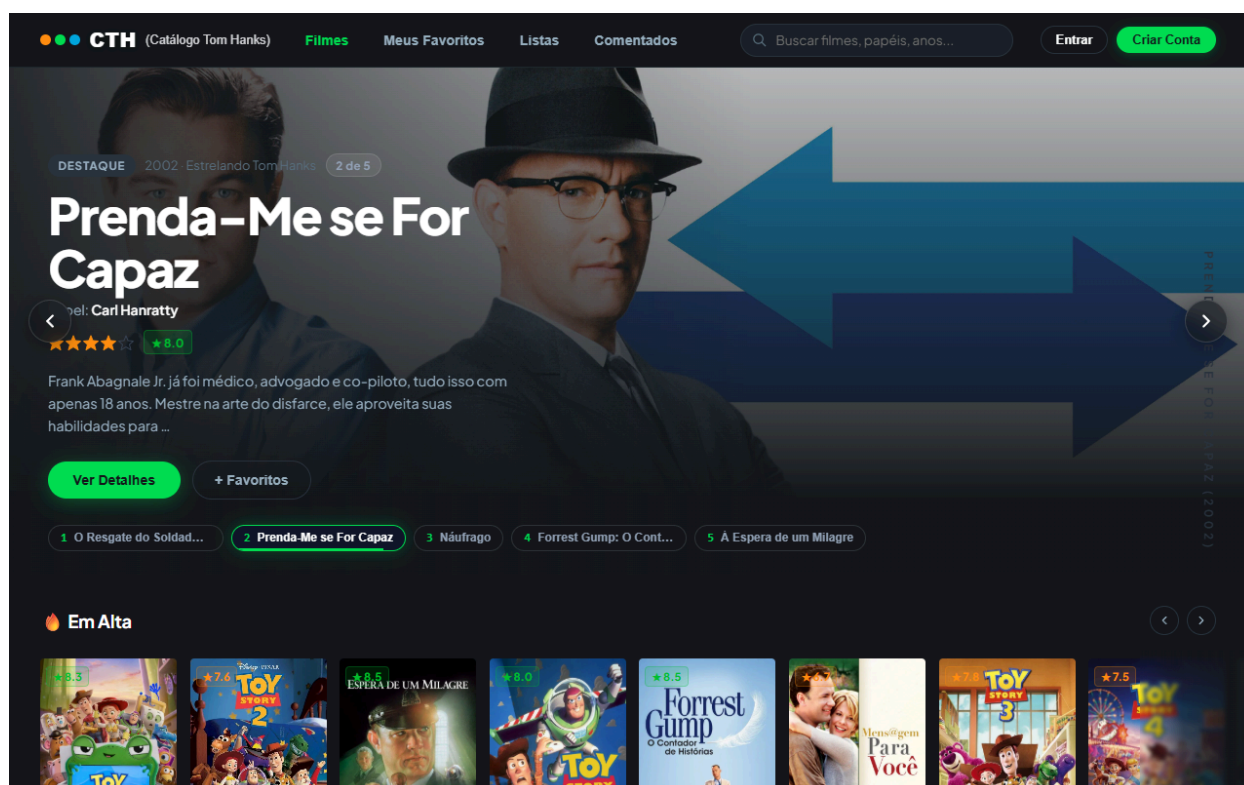


Figura 4: Atividade 2 — interface do catálogo cinematográfico carregando pôsteres dinâmicos da API TMDb

**Dificuldades e como foram resolvidas.** A API do TMDb impõe limite de taxa de requisições (rate limiting) e paginação de dados. Para contornar lentidão na renderização da filmografia completa de mais de 60 obras do ator, implementou-se um mecanismo de cache em memória no backend da aplicação, evitando requisições redundantes a cada recarregamento de página.

### 4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

|                       |                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrição</b>      | Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.                                                                                                                            |
| <b>Data planejada</b> | 28/08/2026                                                                                                                                                                                                      |
| <b>Data realizada</b> | 27/08/2026 20:47                                                                                                                                                                                                |
| <b>Situação</b>       | entregue                                                                                                                                                                                                        |
| <b>Evidência</b>      | GitHub — commit d73c841 + docker-compose.yml + print do login funcionando                                                                                                                                       |
| <b>Link</b>           | <a href="https://github.com/danielmarcalgoncalves/filmecc/commit/d73c841fcd9e2bb6776f7c6cfea25fc5afab212e">https://github.com/danielmarcalgoncalves/filmecc/commit/d73c841fcd9e2bb6776f7c6cfea25fc5afab212e</a> |

**O que foi feito.** Desacoplamento arquitetural da responsabilidade de autenticação, extraindo as rotas de registro, verificação de e-mail com código OTP, login e recuperação de senha para um serviço independente denominado auth-service. A orquestração foi estabelecida via docker-compose.yml, configurando uma rede interna bridge isolada na qual o auth-service não expõe portas públicas ao host, comunicando-se exclusivamente com o gateway/BFF da aplicação principal via HTTP interno e com o MariaDB na porta 3306. Implementou-se criptografia de senhas com bcrypt (fator de custo 10) e geração de tokens stateless JWT assinados digitalmente. Adicionalmente, integrou-se envio transacional de e-mails de confirmação através do provedor Brevo SMTP.

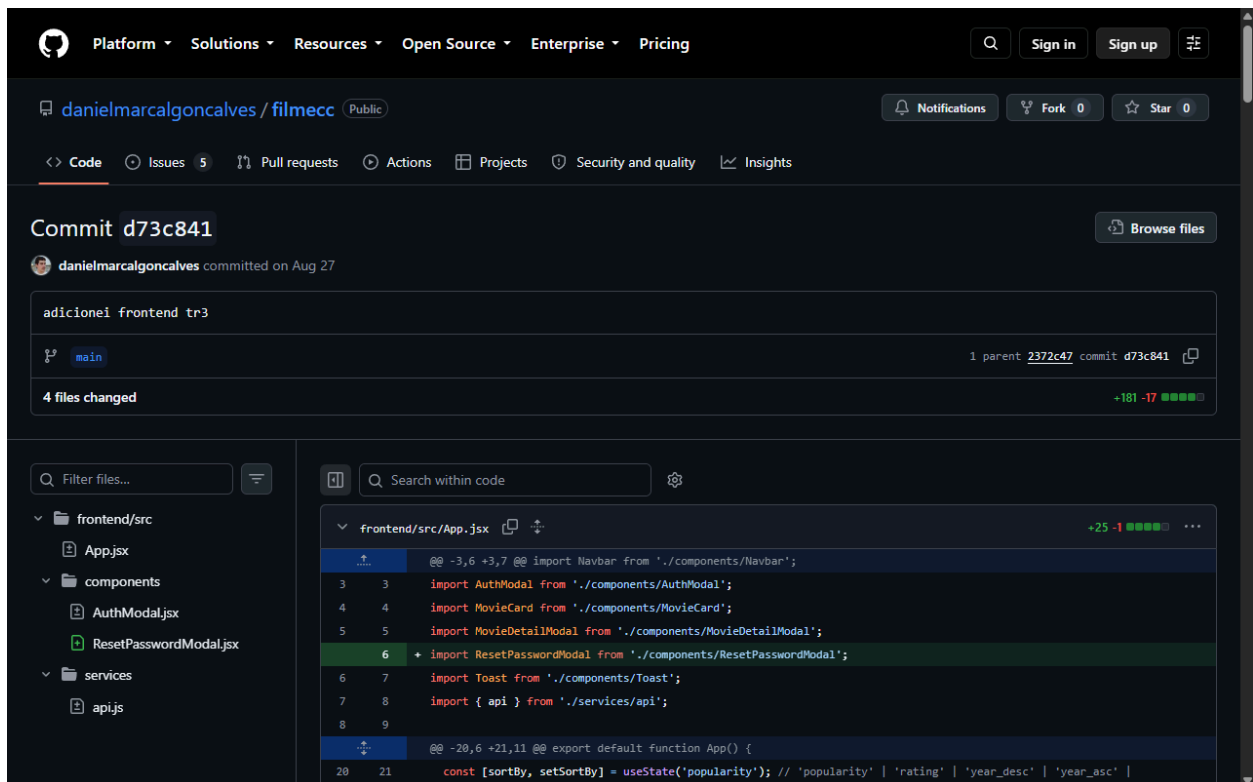


Figura 5: Atividade 3 — commit d73c841 registrado no GitHub em 27/08/2026 com desacoplamento no Docker

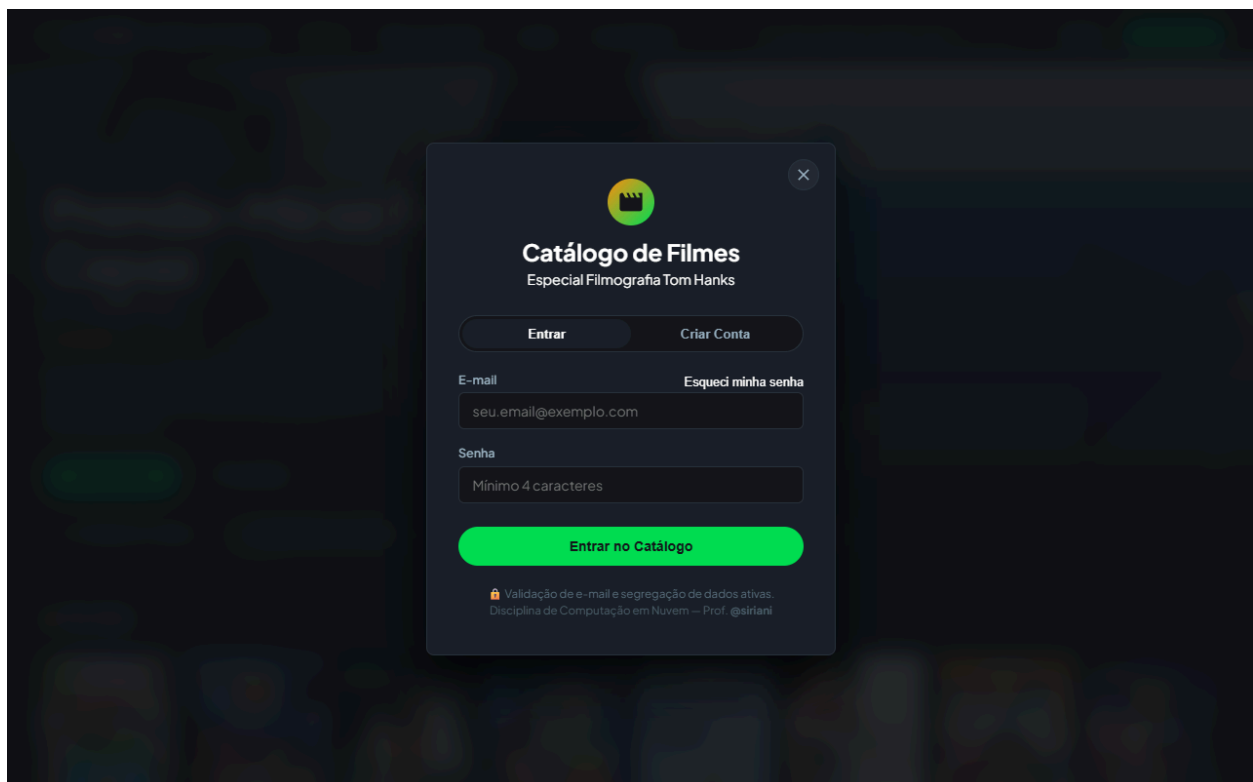


Figura 6: Atividade 3 — modal de autenticação desacoplado com suporte a login, criação de conta e recuperação

**Dificuldades e como foram resolvidas.** A comunicação entre contêineres no Docker Compose falhava inicialmente quando o serviço tentava acessar localhost em vez do nome do serviço registrado no DNS interno do Docker. O problema foi sanado parametrizando a variável `AUTH_SERVICE_URL=http://auth-service:3000` via variáveis de ambiente.

#### 4.4 Atividade 4 — Controle de acesso por papel — RBAC

|                |                                                                                                                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrição      | O campo role passa a decidir permissões reais no backend (403 para usuário comum).                                                                                                                              |
| Data planejada | 04/09/2026                                                                                                                                                                                                      |
| Data realizada | 04/09/2026 14:59                                                                                                                                                                                                |
| Situação       | entregue                                                                                                                                                                                                        |
| Evidência      | GitHub — commit e02ba5a + print da ação e painel de admin com papéis                                                                                                                                            |
| Link           | <a href="https://github.com/danielmarcalgoncalves/filmecc/commit/e02ba5a31390e2cba2e708656697f283a1265877">https://github.com/danielmarcalgoncalves/filmecc/commit/e02ba5a31390e2cba2e708656697f283a1265877</a> |

**O que foi feito.** Implementação de controle rigoroso de acesso baseado em papéis (Role-Based Access Control - RBAC). Foram estabelecidas três categorias de privilégios: Usuário Comum (usuario), Usuário VIP (premium) e Administrador do Sistema (admin). No backend, foram criados middlewares de autorização que interceptam as requisições, validam a role contida no payload verificado do JWT e consultam o estado atual no MariaDB para impedir brechas do tipo Insecure Direct Object References (IDOR). O painel de administração permite que administradores promovam ou rebaixem permissões de usuários, moderem resenhas e visualizem métricas. Usuários comuns que tentam acessar rotas restritas recebem formalmente o status HTTP 403 Forbidden.

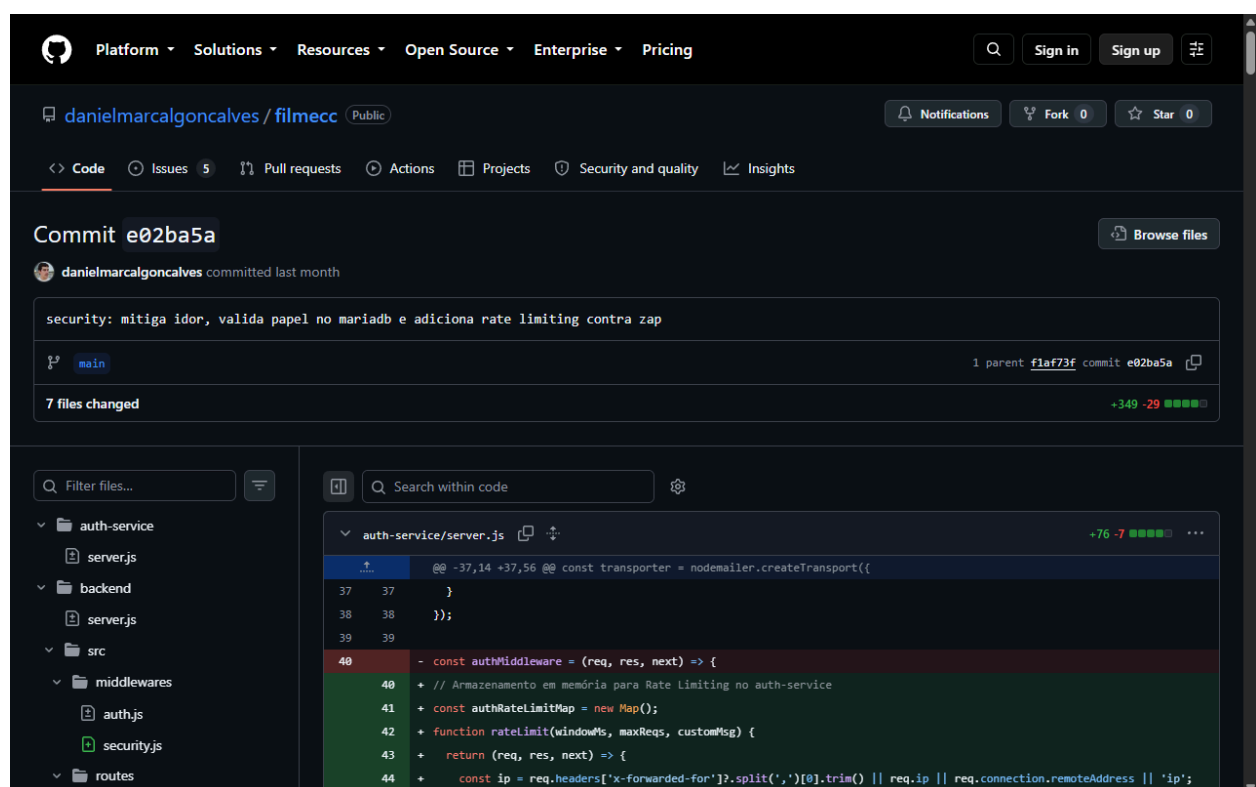


Figura 7: Atividade 4 — commit e02ba5a no GitHub em 04/09/2026 com validação de papéis e segurança

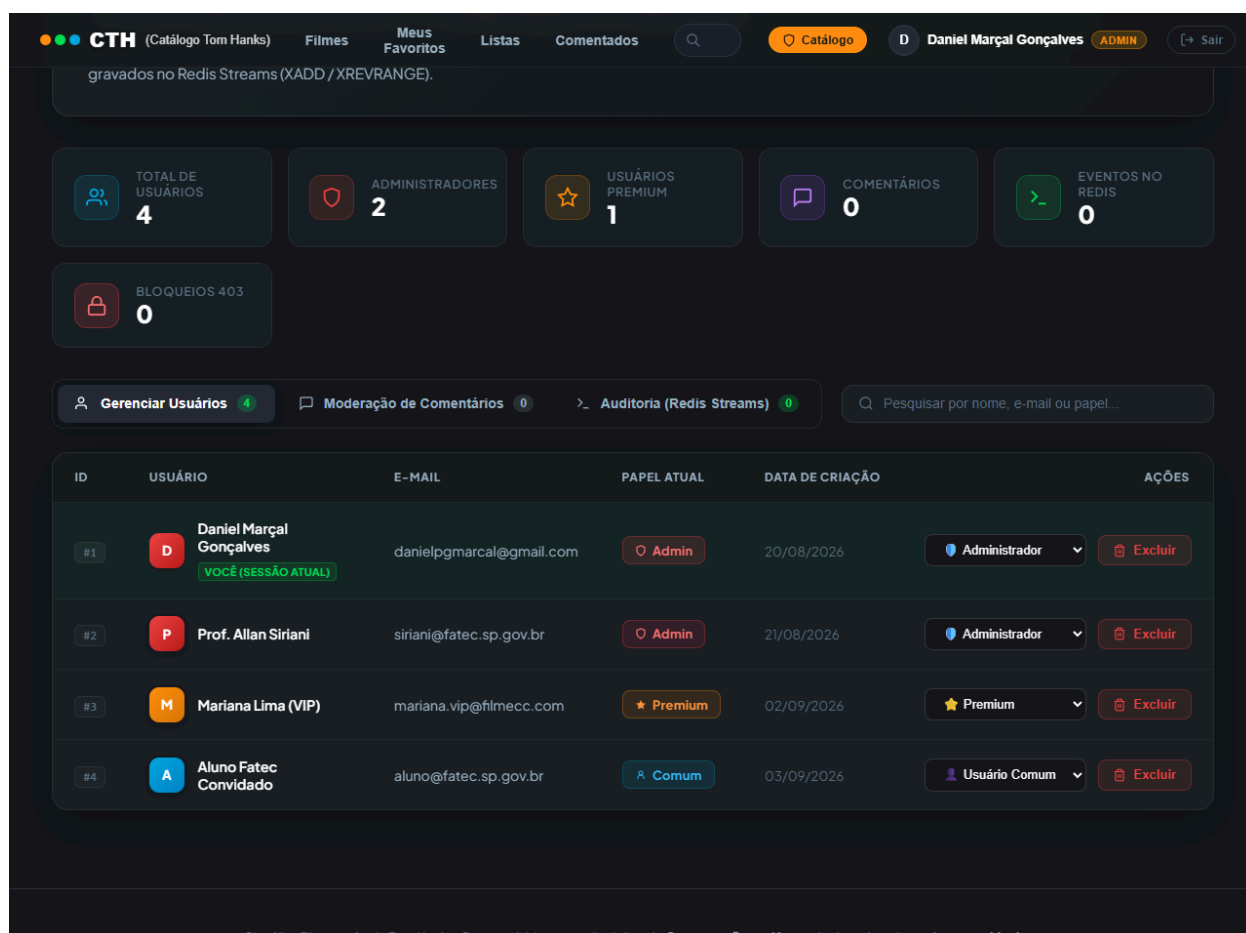


Figura 8: Atividade 4 — Centro Administrativo com gerenciamento de papéis RBAC e bloqueios 403

**Dificuldades e como foram resolvidas.** O risco de escalonamento indevido de privilégios por manipulação de estado no cliente (frontend). A resolução foi garantir que o frontend apenas reflita a interface de acordo com o token verificado, e que toda e qualquer mutação de perfil no backend seja estritamente checada na camada de banco de dados (`papel == 'admin'`), rejeitando imediatamente qualquer requisição não autorizada.

#### 4.5 Atividade 5 — Logs e auditoria

|                       |                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Descrição</b>      | Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.                                                                                                                             |
| <b>Data planejada</b> | 25/09/2026                                                                                                                                                                                                      |
| <b>Data realizada</b> | 08/09/2026 11:21                                                                                                                                                                                                |
| <b>Situação</b>       | entregue                                                                                                                                                                                                        |
| <b>Evidência</b>      | GitHub — commit fb46664 + print da consulta de logs pelo admin via Redis Streams                                                                                                                                |
| <b>Link</b>           | <a href="https://github.com/danielmarcalgoncalves/filmecc/commit/fb4666424d959184d7d32457da1bf3cd7a09d9f8">https://github.com/danielmarcalgoncalves/filmecc/commit/fb4666424d959184d7d32457da1bf3cd7a09d9f8</a> |

**O que foi feito.** Construção de um microserviço dedicado de ingestão e consulta de auditoria (log-service) apoiado em **Redis Streams**. Compreendendo que eventos de auditoria possuem um perfil de escrita intensiva e append-only (write-heavy), optou-se tecnicamente por não onerar o banco relacional

MariaDB com locks de gravação contínua. Para a ingestão, o serviço utiliza o comando `XADD logs:audit *`, gerando identificadores temporais nativos com precisão de milissegundos UTC. Para a visualização dos administradores no painel web, o endpoint executa `XREVRANGE logs:audit + - COUNT 100`, obtendo os eventos em ordem cronológica inversa com altíssima velocidade. O sistema audita logins com sucesso, tentativas de acesso inválidas, inclusão de favoritos, uploads e bloqueios 403 Forbidden.

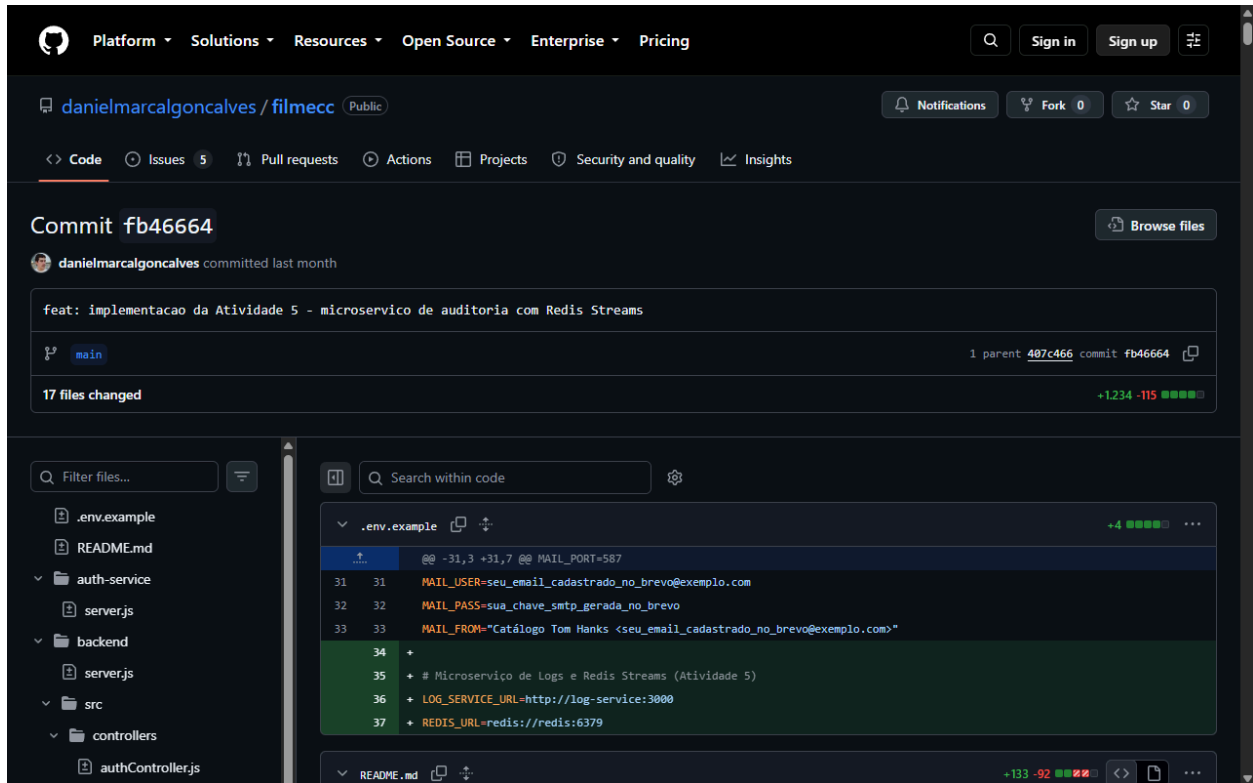


Figura 9: Atividade 5 — commit fb46664 no GitHub em 08/09/2026 com arquitetura Redis Streams

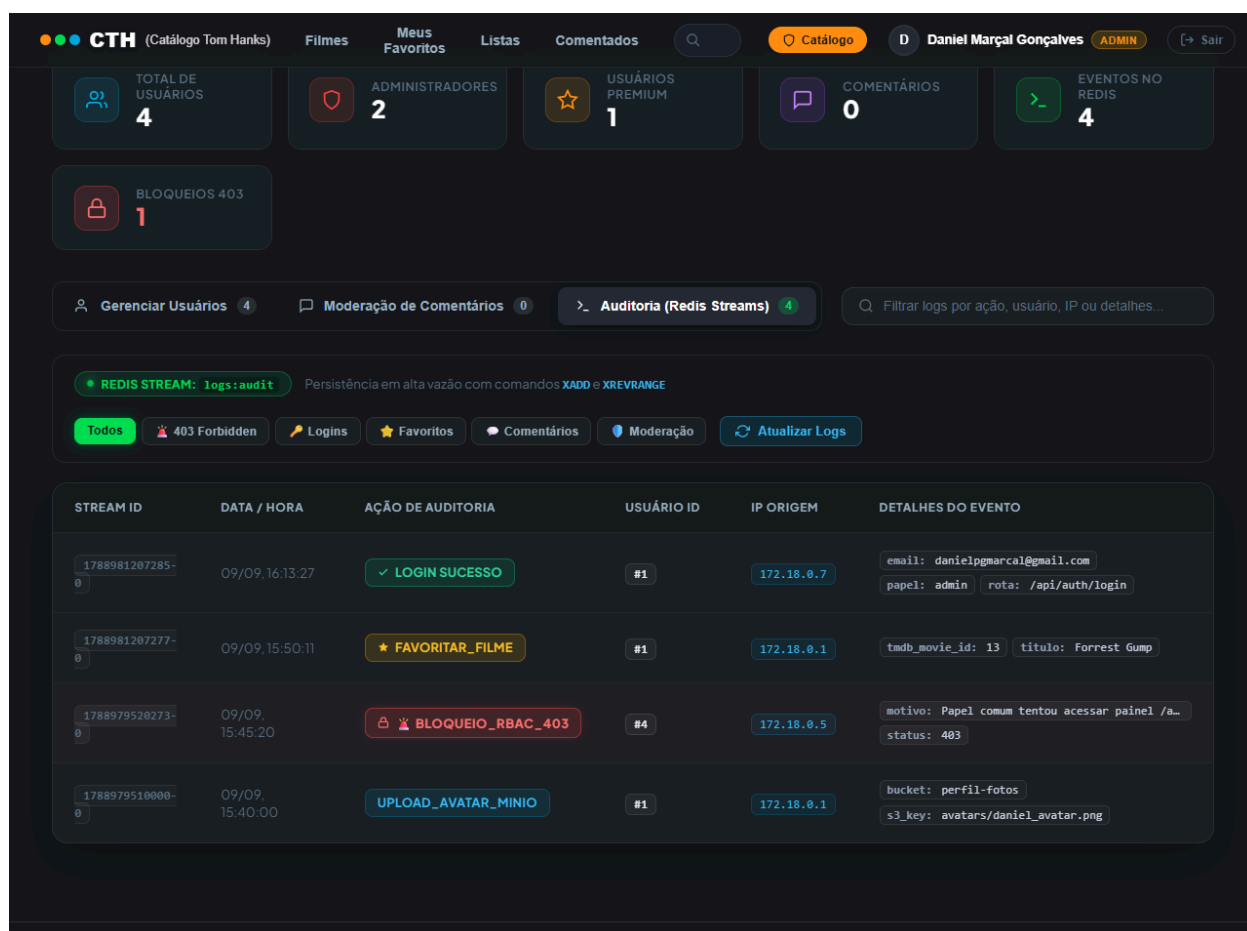


Figura 10: Atividade 5 — Painel de Auditoria em tempo real consultando eventos no Redis Streams

**Dificuldades e como foram resolvidas.** Garantir que falhas de conectividade ou lentidão eventual no envio de logs não interrompessem a experiência do usuário final nos outros serviços. A solução consistiu na criação de uma rotina assíncrona desacoplada com `AbortController` e tempo limite estrito (timeout de 2,5s), garantindo que a gravação do log ocorra em segundo plano sem bloquear a resposta HTTP principal.

#### 4.6 Atividade 6 — Upload e perfil de usuário

|                |                                                                                                                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descrição      | Página de perfil com avatar no MinIO; só a referência fica no banco relacional.                                                                                                                                 |
| Data planejada | 02/10/2026                                                                                                                                                                                                      |
| Data realizada | 09/09/2026 11:54                                                                                                                                                                                                |
| Situação       | entregue                                                                                                                                                                                                        |
| Evidência      | GitHub — commit c967db2 + print do perfil com foto carregada do MinIO                                                                                                                                           |
| Link           | <a href="https://github.com/danielmarcalgoncalves/filmecc/commit/c967db27284846558571ea1ec6a3bd8715b679a8">https://github.com/danielmarcalgoncalves/filmecc/commit/c967db27284846558571ea1ec6a3bd8715b679a8</a> |

**O que foi feito.** Implementação da camada de armazenamento de arquivos não estruturados através da integração com o **MinIO**, um servidor de Object Storage de alta performance compatível com a API S3 da AWS. Foi desenvolvida a interface de perfil do usuário (`ProfilePage.jsx`), permitindo alteração de nome, biografia e upload de imagem de avatar. O backend recebe o fluxo binário via multipart/form-data, valida o tipo MIME e o tamanho máximo do arquivo, realiza o upload para o bucket seguro `perfil-fotos` no MinIO e grava no MariaDB estritamente a chave do objeto (referência textual), respeitando as boas

práticas de arquitetura em nuvem que desaconselham o armazenamento de arquivos binários em colunas relacionais do tipo BLOB.

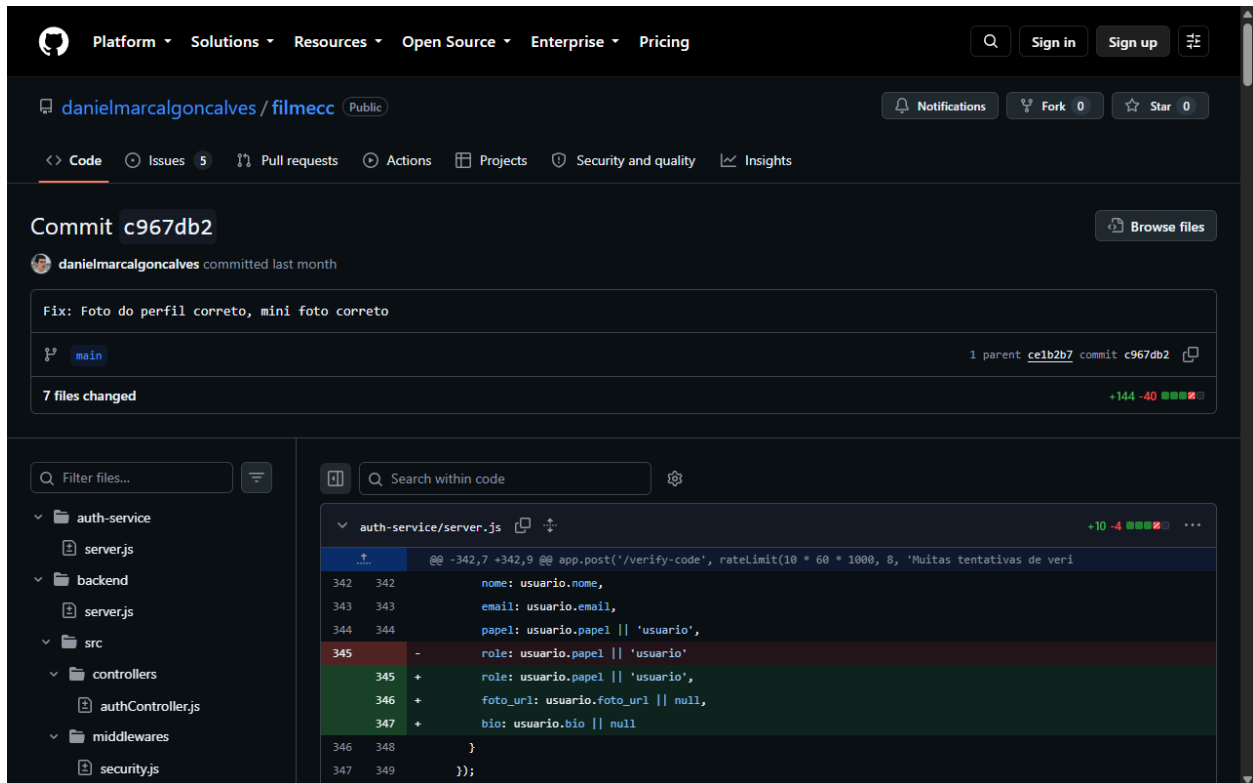


Figura 11: Atividade 6 — commit c967db2 no GitHub em 09/09/2026 com upload de avatar no MinIO S3

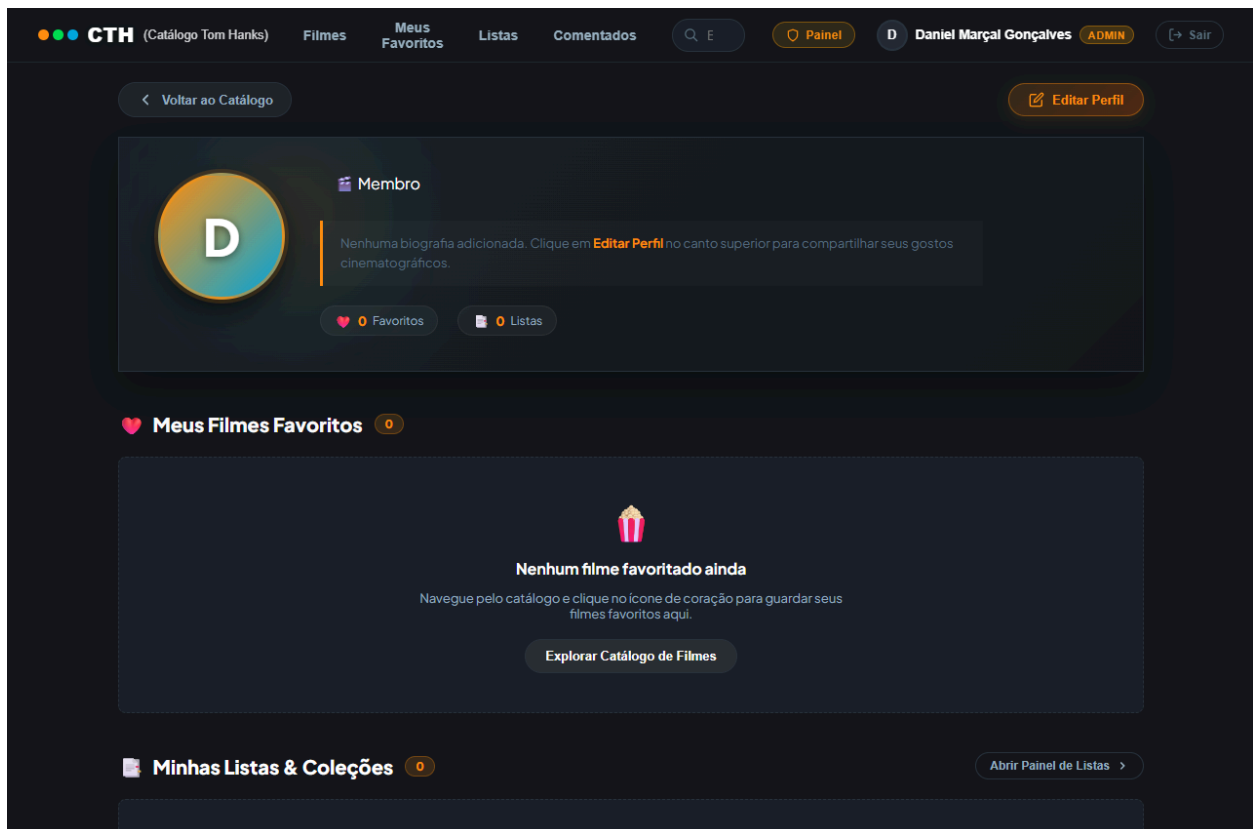


Figura 12: Atividade 6 — tela de perfil do usuário com foto de perfil e dados carregados

**Dificuldades e como foram resolvidas.** Conflito de portas entre o host e contêineres pré-existentes na porta padrão 9000 do MinIO. A dificuldade foi resolvida mapeando portas dedicadas no Docker Compose (9010:9000 para a API S3 e 9011:9001 para o Console Web administrativo do MinIO), além de implementar streaming de imagens com fallback visual gracioso quando a foto não está configurada.

## 5 Considerações finais

O percurso prático trilhado ao longo deste primeiro bimestre proporcionou uma sólida e transformadora base conceitual em computação em nuvem. A experiência de vivenciar na prática a transição arquitetural — partindo de um monólito didático em Flask até atingir uma malha de serviços containerizados desacoplados — consolidou a compreensão de conceitos fundamentais como isolamento de ambiente, persistência poliglota especializada e segurança orientada a papéis.

Dentre as principais dificuldades superadas, destacam-se a orquestração precisa das redes internas de contêineres sem vazamento desnecessário de portas para a máquina hospedeira, a harmonização do protocolo de mensageria assíncrona via Redis Streams para evitar gargalos em bancos relacionais, e a correta segregação de credenciais e segredos em arquivos de ambiente não versionados. Caso o projeto fosse reiniciado, uma decisão arquitetural que seria antecipada envolveria a padronização de contratos de API via Swagger/OpenAPI desde o primeiro microserviço, o que agilizaria ainda mais a integração entre as equipes de frontend e backend.

Como perspectiva para o segundo bimestre da disciplina, pretendo aprofundar os conhecimentos em infraestrutura como código (IaC), esteiras de integração e entrega contínua (CI/CD) com GitHub Actions e a transposição destes contêineres para plataformas de nuvem pública gerenciadas em modelos PaaS e IaaS.

## 6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

---

Daniel Marçal Gonçalves

---

Pompeia, 07/10/2026